Shoes vs Sandals: Image Classification with a Self-Built CNN and Transfer Learning

Oct 31, 2026 · @Danial

Muhammad Danial Khalid, BSCS, Superior University, Lahore. Deep Learning course, Dr. Munir Ahmad.

A MobileNetV2 model fine-tuned with transfer learning classifies single footwear photos as shoe or sandal with 72.5% accuracy (29 of 40 held-out photos), while a CNN trained from scratch on the same self-collected dataset reaches only 47.5%. Both models run in the browser on the deployed website.

1. Problem statement

Given one photo of footwear, the model decides whether it shows a **shoe** or a **sandal**. This is a binary image classification task.

- **Shoe:** closed footwear, such as sneakers, joggers, formal shoes, boots and loafers.
- **Sandal:** any open style, including sandals, chappals, Peshawari chappal, slippers and flip-flops.

The project compares two ways of training a deep network on a small dataset collected by the author: a convolutional network trained from random weights (self learning), and a pretrained MobileNetV2 fine-tuned on the same data (transfer learning). A footwear classifier is a building block for shop inventory tagging, visual search in online stores, and sorting at return counters.

2. Dataset

The dataset is primary data, collected for this project rather than downloaded from Kaggle. It holds 2,184 training images and two separate test sets.

Sources

- **Videos** recorded at shoe shops and footwear racks, one class per video: 32 shoe videos and 42 sandal videos. Frames were sampled every 0.5 seconds, and blurry and near-duplicate frames were dropped automatically.
- **Single photos** of individual pairs, contributed by family, friends and colleagues through a bilingual (English and Urdu) Google Form that asked for consent and explained how the photos would be used.

Privacy: faces and personal details were excluded, phone metadata (GPS location, device, time) was stripped, and file names were anonymised before publishing.

Split	Shoes	Sandals	What it contains
Train	1,175	1,009	Frames from 31 + 41 videos, plus 53 + 44 single photos
Test: video frames	13	12	One whole video per class, never seen in training
Test: single photos	20	20	Single photos of pairs not used in training

Leakage control: frames from the same video are near-copies of each other. Every video was therefore placed whole in either train or test, never split across both. Otherwise, the test accuracy would measure memory, not generalisation.

3. Preprocessing and augmentation

Every image goes through the same four steps in training, in `predict.py` and on the website, so the models always see input in the same form.

1. Rotate phone photos upright using their EXIF orientation tag.
2. Convert to RGB and resize to 224 x 224 pixels (no cropping).
3. Scale pixel values from 0..255 to -1..1 with x / 127.5 - 1, the range MobileNetV2 expects.
4. Batch 32 images at a time; decoded images are cached in memory for speed.

Augmentation (training only): random horizontal flip, small rotation, zoom, contrast and brightness changes. Zoom is biased towards zooming in, so a rack of shoes looks more like a few shoes, closer to the single photos users upload. The self-built CNN uses milder augmentation (rotation 0.04, zoom up to 25%) than the transfer model (rotation 0.08, zoom up to 45%), because a network learning from zero struggled with the stronger version.

Class and source balance: the training set holds about 2,100 video frames but only 97 single photos. Each single photo is weighted 5 times a video frame in the loss, and the two classes are reweighted to contribute equally.

4. Model architectures

Both models take a 224 x 224 x 3 image and output one sigmoid value: the probability that the photo is a shoe. A value of 0.5 or more means shoe.

	Self-built CNN	Transfer learning (MobileNetV2)
Starting weights	Random	ImageNet (1.4 million images, 1,000 classes)
Feature extractor	4 blocks of Conv2D 3x3 + BatchNorm + ReLU + MaxPool, with 32, 64, 128, 128 filters	MobileNetV2 base: 154 layers of inverted residual blocks
Head	Global average pooling, Dropout 0.4, Dense(1, sigmoid)	Global average pooling, Dropout 0.3, Dense(1, sigmoid)
Parameters	242,017	2,259,265

Why these two: the CNN is small on purpose, so it can learn something from about 2,000 images without memorising them outright. MobileNetV2 was chosen for transfer learning because it is accurate for its size and light enough to run inside a web browser, which the deployment needed.

5. Training setup

Both models were trained in Google Colab on a T4 GPU with TensorFlow and Keras, using binary cross-entropy loss and the Adam optimiser.

	Self-built CNN	Transfer learning
Epochs	40	6 (head only) + 6 (fine-tuning)
Learning rate	1e-3, cosine decay to 0	1e-3 for the head, then 1e-5
Trainable layers	All	Stage 1: new head only. Stage 2: head + last 30 MobileNetV2 layers
Augmentation	Mild	Strong

The number of epochs was fixed in advance. The video-frame test set was plotted during training for the curves only and was not used to choose epochs or settings, so the reported test accuracy is not tuned on the test data.

The complete pipeline (data loading, preprocessing, both models, evaluation, saving the weights and exporting to TensorFlow.js) is in one notebook in the repository: `notebook/shoes_vs_sandals_training.ipynb`.

6. Results

Transfer learning gets 29 of 40 single photos right (72.5%); the self-built CNN gets 19 of 40 (47.5%), which is no better than chance. The single-photo test set is the honest measure, because single photos are what people upload to the website.

Model	Video frames (n = 25)	Single photos (n = 40)
Transfer learning, MobileNetV2	25 / 25 (100%)	29 / 40 (72.5%)
Self-built CNN	17 / 25 (68%)	19 / 40 (47.5%)

Where the errors are (single photos):

Model	Sandals correct	Sandals called shoe	Shoes correct	Shoes called sandal
Transfer learning	14	6	15	5
Self-built CNN	13	7	6	14

The transfer model's mistakes are spread across both classes, and most of them sit near the 0.5 boundary (probabilities between 0.31 and 0.73), so the model is unsure rather than confidently wrong. The CNN calls most shoes sandals: it has not learned what separates the two classes on single items.

Read the numbers as rough. With 40 test photos, one image moves accuracy by 2.5 points. Two training runs of the same notebook gave 82.5% and 72.5% for the transfer model, so its true single-photo accuracy is roughly in the 70 to 85% range. Training curves and confusion-matrix charts are on the website and in `results/` in the repository.

7. Discussion and limitations

Transfer learning wins because the hard part of vision, recognising edges, textures and shapes, was already learned from ImageNet. With about 2,000 images, the new head and the last 30 layers only had to learn which of those features separate an open sandal from a closed shoe.

The self-built CNN had to learn those features from nothing. Its training accuracy stalled around 76% in a first run and its single-photo accuracy stayed near chance. The likely cause is the data mix: 95% of training images are frames of racks and shelves, so the cheapest pattern to learn was the look of a shop scene, not the footwear.

The same data mix limits the transfer model. It is perfect on video frames (25 of 25) but drops to 72.5% on single photos, a clear sign of a gap between the training data and real uploads.

Limitations

- Only 97 single photos in training, compared with about 2,100 rack frames.
- Small test sets (25 and 40 images), so accuracy has wide uncertainty.
- Many single photos come from the same few contributors, so test pairs may resemble training pairs in style and setting.
- Hard borderline styles: Peshawari chappal and closed-toe sandals look like shoes, and are labelled sandals by the rule used here.
- A few photos show the footwear being worn, with legs or clothing in the frame.

8. Deployment

The website is live at shoes-vs-sandals.vercel.app. It is a static page hosted on Vercel: both models were converted to TensorFlow.js and run inside the visitor's browser, so no server is needed and uploaded photos never leave the device. A switch lets the visitor compare the two models on the same photo, and sample test photos can be tried with one click. I checked that the browser predictions match the Python models (for example 0.330 in the browser against 0.339 in Python for the same photo).

The GitHub repository holds:

- `models/` : the final trained weights of both models. The transfer weights are stored in 3 parts because GitHub's upload page limits file size.
- `predict.py` : rebuilds both architectures, loads the weights and classifies a photo or a whole folder, printing accuracy for labelled folders.
- `notebook/` : the full training notebook. `scripts/` : the frame-extraction script.
- `test_samples/` : 39 held-out test photos. `results/` : curves, confusion matrices, results.json.
- `README.txt` : folder map, run commands, dataset, training and results summary.

Run the models locally with `pip install -r requirements.txt` , then `python predict.py test_samples --model transfer` . The full dataset is on Google Drive.

9. Conclusion and future work

On a small self-collected dataset, transfer learning is the practical choice: MobileNetV2 reached 72.5% on unseen single photos and 100% on unseen video frames, while a CNN trained from scratch did not generalise. The main lesson is about data, not architecture:

the training set should look like what the model will see in use.

Next steps, in order of expected impact:

1. Collect 300 or more single photos per class from many different pairs and places.
2. Crop individual shoes out of rack frames with a pretrained footwear detector, to turn rack footage into single-item examples.
3. Grow both test sets so accuracy can be measured to within a few points.
4. Add an "unsure" answer on the website when the probability is between 0.4 and 0.6.

10. Links and AI use

	Link
Live website	shoes-vs-sandals.vercel.app
GitHub repository	github.com/mdk-aarbi/shoes-vs-sandals
Full dataset	Google Drive
Presentation video	YouTube link to be added

AI use disclosure: AI assistants (Claude) were used, as permitted for this project, to help write the training notebook, the website, the documentation and this report. The dataset was collected by the author, the models were trained and evaluated by the author in Google Colab, and every number in this report comes from those training runs.